

Introducción al Diseño Digital con HDL y Herramientas EDA de Síntesis

Laboratorio 1 – EL3313 Taller de Diseño Digital

Thomas Reed, Santiago Chavarria, Emanuel Varela, Andres Madrigal
Escuela de Ingeniería Electrónica
Tecnológico de Costa Rica
Email: treed.com

Resumen—El presente documento describe el desarrollo del Laboratorio 1 del curso Taller de Diseño Digital. El objetivo principal del laboratorio es introducir el uso de lenguajes de descripción de hardware (HDL), específicamente Verilog y SystemVerilog, dentro del flujo moderno de diseño digital basado en herramientas EDA.

Se abordan conceptos fundamentales relacionados con familias lógicas TTL y CMOS, parámetros eléctricos característicos, tiempos de propagación, fan-out, disparadores Schmitt, modulación por ancho de pulso (PWM), protocolos de comunicación UART y fundamentos de síntesis lógica para FPGA.

Asimismo, se establece el marco teórico necesario para el desarrollo de sistemas digitales combinatoriales completamente sintetizables, enfatizando el modelado estructural y comportamental, así como la validación mediante simulación y verificación post-síntesis.

I. INTRODUCCIÓN

El diseño digital moderno ha evolucionado desde la implementación directa con compuertas discretas hasta metodologías basadas en lenguajes de descripción de hardware (HDL). Estos lenguajes permiten modelar sistemas electrónicos digitales de forma abstracta, facilitando su simulación, síntesis e implementación en dispositivos programables como FPGAs o en circuitos integrados específicos (ASICs).

A diferencia de los lenguajes de programación tradicionales, los HDLs describen estructuras físicas y comportamiento concurrente de hardware. Esto implica que cada bloque descrito representa circuitos reales que serán implementados físicamente después del proceso de síntesis lógica. Por tanto, el diseñador debe comprender completamente la arquitectura del sistema, incluyendo tablas de verdad, diagramas de bloques y especificaciones temporales.

En este laboratorio se introduce el uso de Verilog para descripción sintetizable y SystemVerilog para verificación. Se desarrollan módulos digitales combinatoriales y secuenciales básicos, que posteriormente son simulados, sintetizados y validados en una FPGA.

El propósito de este documento es establecer el fundamento teórico que sustenta el desarrollo práctico del laboratorio, proporcionando un análisis detallado de las tecnologías digitales involucradas y del flujo de diseño empleado.

II. MARCO TEÓRICO

II-A. Familias Lógicas TTL

La familia TTL (Transistor-Transistor Logic) corresponde a una tecnología basada en transistores bipolares (BJT). Dentro de la serie 74xx existen distintas variantes:

II-A1. 74L: Versión de bajo consumo (Low Power). Presenta menor velocidad de conmutación y menor consumo que la TTL estándar.

II-A2. 74LS: Low Power Schottky. Utiliza diodos Schottky para evitar saturación profunda de los transistores, reduciendo significativamente el tiempo de propagación respecto a la serie L y mejorando eficiencia energética.

II-A3. 74HC: Aunque pertenece tecnológicamente a CMOS, es compatible funcionalmente con la numeración 74xx. Presenta bajo consumo estático y mayor inmunidad al ruido.

Las características típicas de TTL incluyen:

- $V_{CC} = 5V$
- $V_{IL} \leq 0,8V$
- $V_{IH} \geq 2,0V$
- Fan-out típico: 10
- Tiempos de propagación: 10–30 ns

II-B. Familia CMOS 4000

La familia CMOS 4000 utiliza transistores MOSFET complementarios (PMOS y NMOS). Sus principales ventajas incluyen:

- Bajo consumo estático
- Alta impedancia de entrada
- Amplio rango de alimentación (3V–15V)
- Alta inmunidad al ruido

El consumo dinámico depende de la frecuencia de conmutación y la capacitancia de carga.

II-C. Parámetros Eléctricos Fundamentales

II-C1. V_{IL} y V_{IH} : Voltajes máximos y mínimos que garantizan reconocimiento lógico bajo y alto respectivamente.

II-C2. V_{OL} y V_{OH} : Niveles de salida correspondientes a estados lógicos bajo y alto.

II-C3. I_{IK} y I_{OK} : Corrientes asociadas a protección interna y capacidad de conducción de salida.

II-C4. Tiempos de Propagación:

- t_{PD} : Tiempo total de propagación
- t_{PLH} : Bajo a alto
- t_{PHL} : Alto a bajo

II-C5. Tiempos de Transición:

- t_r : Tiempo de subida
- t_f : Tiempo de bajada

II-D. Fan-Out

El fan-out define cuántas entradas estándar puede manejar una salida sin degradar niveles lógicos. En TTL típicamente es 10, mientras que en CMOS puede superar 50 dependiendo de la capacitancia.

II-E. Estructura CMOS de una NAND

Una compuerta NAND CMOS consiste en:

- Dos transistores PMOS en paralelo en la red de pull-up
- Dos transistores NMOS en serie en la red de pull-down

Esta configuración permite bajo consumo estático, ya que no existe camino directo entre VDD y GND en estado estable.

II-F. Pull-Up y Pull-Down

Son resistencias utilizadas para fijar un nivel lógico definido cuando una línea no está activamente conducida. Son fundamentales en entradas flotantes y buses compartidos.

II-G. Disparador Schmitt

Un disparador Schmitt introduce histéresis mediante dos umbrales distintos de conmutación, mejorando inmunidad al ruido y eliminando rebote en señales mecánicas. El 74*14 es un inversor con disparador Schmitt integrado.

II-H. Modulación por Ancho de Pulso (PWM)

La modulación PWM consiste en variar el ciclo de trabajo de una señal periódica manteniendo frecuencia constante. El valor promedio de voltaje es proporcional al duty cycle:

$$V_{avg} = D \cdot V_{max} \quad (1)$$

donde D es el ciclo de trabajo.

II-I. Rebote y Debouncing

Los interruptores mecánicos generan múltiples transiciones al activarse. El debouncing puede realizarse mediante:

- Filtros RC
- Disparadores Schmitt
- Lógica secuencial con temporización

II-J. Modelado Estructural vs Comportamental

Modelado estructural: describe interconexión explícita de módulos.

Modelado comportamental: describe funcionamiento mediante ecuaciones o bloques always.

Ejemplo comportamental en Verilog:

```
assign y = a & b;
```

II-K. Síntesis Lógica

La síntesis convierte código HDL en una red de compuertas equivalente (netlist). En FPGA, el diseño se implementa en LUTs y flip-flops configurables. En ASIC, se mapea a celdas estándar de una biblioteca tecnológica.

II-L. Tecnología FPGA

Una FPGA contiene:

- LUTs (Look-Up Tables)
- Flip-flops
- Bloques de memoria
- Rutas programables
- Bloques de I/O

Su configuración se realiza mediante un bitstream generado por herramientas EDA.

II-M. Protocolo UART

UART (Universal Asynchronous Receiver/Transmitter) es un protocolo serial asíncrono que utiliza:

- 1 bit de inicio
- 8 bits de datos
- Opcionalmente bit de paridad
- 1 o 2 bits de parada

La transmisión TX envía los bits de forma secuencial a una velocidad determinada (ej. 9600 baudios), utilizando temporización basada en un divisor de frecuencia.

III. METODOLOGÍA DE DISEÑO

El desarrollo del presente laboratorio se realizó siguiendo un flujo de diseño digital estructurado, utilizando la tarjeta de desarrollo Nexys 4 DDR y la herramienta Xilinx Vivado Design Suite.

El proceso seguido para cada ejercicio fue el siguiente:

III-A. Especificación del Sistema

Cada problema planteado fue analizado para traducir los requerimientos funcionales en especificaciones digitales formales. Se identificaron entradas, salidas, restricciones de temporización y naturaleza del circuito (combinacional o secuencial).

III-B. Modelado en HDL

Los módulos fueron descritos en Verilog utilizando modelado comportamental sintetizable. En el caso de lógica combinacional, se emplearon asignaciones continuas mediante la instrucción `assign`, garantizando ausencia de elementos de memoria implícitos.

Se verificó que las descripciones fueran completamente sintetizables y libres de construcciones no soportadas por síntesis.

III-C. Simulación Comportamental

Se desarrollaron bancos de prueba (testbench) en Verilog/SystemVerilog para validar funcionalmente cada diseño antes de la síntesis. Las simulaciones fueron ejecutadas en Vivado, verificando:

- Correcta funcionalidad lógica.
- Ausencia de errores o advertencias.
- Cobertura de casos representativos.

III-D. Síntesis Lógica

Posteriormente, se ejecutó la síntesis en Vivado para generar el netlist correspondiente. Se revisó que:

- No se generaran latches.
- No se infirieran flip-flops en circuitos combinacionales.
- La implementación utilizara únicamente LUTs.

III-E. Implementación en FPGA

Se realizó la asignación de pines mediante archivo de restricciones (.xdc), asociando switches, LEDs, reloj, botón y líneas de comunicación serial de la Nexys 4 DDR. Finalmente, se generó el bitstream y se validó el funcionamiento en hardware.

IV. DESARROLLO

IV-A. Ejercicio 1 – Complemento a 2

IV-A1. Especificación: Se diseñó un módulo combinacional que recibe como entrada un vector de 4 bits proveniente de los switches físicos de la FPGA y produce como salida su equivalente en complemento a 2, el cual es mostrado en los LEDs de la tarjeta.

IV-A2. Fundamento de Operación: El complemento a 2 de un número binario se obtiene invirtiendo todos sus bits y sumando una unidad. La operación implementada fue:

$$C2(x) = \bar{x} + 1 \quad (2)$$

Para un número de 4 bits, esta representación permite trabajar con valores con signo dentro del rango -8 a 7 .

SW	LED (C2)
0000	0000
0001	1111
0010	1110
0011	1101
0100	1100
0101	1011
0110	1010
0111	1001
1000	1000
1001	0111
1010	0110
1011	0101
1100	0100
1101	0011
1110	0010
1111	0001

Cuadro I

TABLA DE VERDAD DEL COMPLEMENTO A 2 EN 4 BITS.

IV-A3. Tabla de Verdad:

IV-A4. Implementación: El módulo fue descrito mediante asignación continua, lo cual representa directamente una operación combinacional sin necesidad de bloques secuenciales. La expresión utilizada fue equivalente a una inversión bit a bit seguida de una suma de una unidad.

Desde el punto de vista de hardware, la síntesis interpreta este diseño como una red de inversores y un sumador de 4 bits. Debido a la naturaleza del problema, no se requieren relojes, registros ni memoria.

IV-A5. Validación: El testbench aplicado evaluó distintas combinaciones de entrada y permitió comprobar visualmente que la salida correspondía al complemento a 2 esperado. Además, la implementación en la FPGA confirmó el comportamiento correcto al modificar el estado de los switches.

IV-B. Ejercicio 2 – Multiplexor 4:1 Parametrizable

IV-B1. Especificación: Se implementó un multiplexor 4:1 parametrizable, capaz de seleccionar una entre cuatro entradas de tamaño configurable mediante una señal de selección de 2 bits. El diseño fue validado para anchos de 4, 8 y 16 bits.

IV-B2. Fundamento de Operación: La función del multiplexor consiste en enrutar hacia la salida una de las cuatro entradas disponibles según el valor del selector:

- `sel = 00` selecciona `in0`
- `sel = 01` selecciona `in1`
- `sel = 10` selecciona `in2`
- `sel = 11` selecciona `in3`

La parametrización mediante `WIDTH` permitió reutilizar la misma arquitectura para diferentes tamaños de bus, mejorando la escalabilidad del diseño.

IV-B3. Implementación: La lógica se implementó mediante un bloque `always_comb` y una estructura `case`, lo cual garantiza comportamiento puramente combinacional. Se incluyó una asignación por defecto a cero para reforzar la completitud de la lógica y evitar inferencia de latches.

Desde la perspectiva de síntesis, Vivado interpreta esta estructura como una red de multiplexores por cada bit del bus de salida.

IV-B4. Validación: El testbench instanció tres versiones del multiplexor, para 4, 8 y 16 bits, respectivamente. Para cada una se generaron 50 conjuntos de datos aleatorios y se probaron las cuatro selecciones posibles. A diferencia del ejercicio anterior, en este caso se utilizó verificación automática mediante `$fatal`, lo cual permitió detectar de forma inmediata cualquier discrepancia entre la salida esperada y la obtenida.

Este enfoque de prueba fortaleció la confiabilidad del módulo y demostró que la parametrización no afectó su funcionamiento lógico.

IV-C. Ejercicio 3 – Generador PWM

IV-C1. Especificación: Se desarrolló un generador PWM cuyo ciclo de trabajo depende del valor presente en cuatro switches de la tarjeta. La salida fue conectada a un LED, de forma que el brillo percibido variara según el duty cycle seleccionado.

IV-C2. Fundamento de Operación: La modulación por ancho de pulso se basa en mantener una frecuencia aproximadamente constante mientras se modifica la duración del tiempo en alto dentro de cada periodo. En este caso, el periodo fue definido por un contador síncrono y el duty cycle fue calculado a partir del valor de los switches.

La expresión empleada fue:

$$duty = \frac{SW \cdot PERIOD}{16} \quad (3)$$

Esto permitió dividir el rango de control en 16 niveles distintos, correspondientes a los valores posibles de una entrada de 4 bits.

IV-C3. Implementación: El módulo utilizó un contador secuencial de 17 bits que avanza con cada flanco positivo del reloj de 100 MHz. Cuando el contador alcanza el valor máximo del periodo, este se reinicia. Paralelamente, el contador se compara continuamente con el valor calculado del duty cycle para decidir si la salida LED debe permanecer en alto o en bajo.

A diferencia de los dos ejercicios anteriores, este diseño sí es secuencial, ya que depende explícitamente del reloj y utiliza registros internos para almacenar el estado del contador y la salida.

IV-C4. Validación: El testbench generó un reloj artificial y recorrió los 16 valores posibles de la entrada SW, permitiendo observar el cambio progresivo del ancho de pulso. En la FPGA, el efecto se validó visualmente mediante la variación del brillo del LED.

IV-D. Ejercicio 4 – Comunicación UART

IV-D1. Especificación: Se implementó un sistema UART completo capaz de transmitir un mensaje al presionar un botón y, adicionalmente, recibir caracteres desde una terminal serial externa para mostrarlos en los LEDs de la FPGA.

IV-D2. Arquitectura General: Como se muestra en la Figura 1, el sistema está compuesto por un bloque de transmisión activado por botón y un bloque de recepción encargado de decodificar los datos seriales provenientes de la terminal.

IV-D3. Temporización del Sistema: El sistema fue configurado para operar a 9600 baudios usando el reloj principal de 100 MHz de la tarjeta Nexys 4 DDR. Para ello se definió:

$$CLKS_PER_BIT = \frac{100\,000\,000}{9600} \approx 10417 \quad (4)$$

Este parámetro determina cuántos ciclos de reloj corresponden a la duración de cada bit transmitido o recibido, garantizando sincronización temporal adecuada con la terminal serial.

IV-D4. Transmisión UART: La transmisión se implementó mediante el módulo `uart_tx`, el cual utiliza una máquina de estados finitos para enviar una trama serial en formato 8N1. La secuencia incluye:

- Estado de reposo
- Bit de inicio
- Ocho bits de datos
- Bit de parada
- Etapa de limpieza

El mensaje transmitido fue “Hola mundo”, complementado con retorno de carro y salto de línea. El módulo `hello_sender` administra este proceso mediante una FSM que recorre los caracteres del mensaje y activa la señal de transmisión byte por byte.

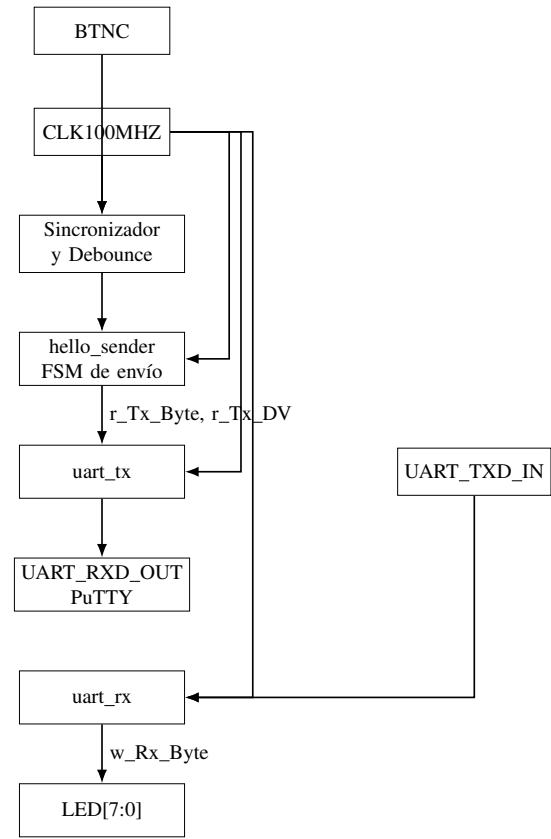


Figura 1. Diagrama de bloques del sistema UART implementado en la FPGA Nexys 4 DDR.

IV-D5. Debounce y Sincronización del Botón: Para evitar múltiples activaciones indeseadas causadas por el rebote mecánico del botón, se implementó un bloque de debounce basado en conteo temporal. Además, la señal fue sincronizada respecto al reloj del sistema antes de utilizarse como evento de disparo. Esto permitió que una sola pulsación generara un único envío del mensaje completo, sin necesidad de ejercer presión adicional sobre el botón.

IV-D6. Recepción UART: La recepción se implementó con el módulo `uart_rx`, también basado en una máquina de estados finitos. El diseño detecta el bit de inicio y muestrea la línea serial en instantes centrados dentro de cada bit para mejorar robustez frente a errores temporales.

Adicionalmente, la señal serial de entrada fue pasada por un sincronizador de dos etapas, reduciendo el riesgo de metastabilidad al ingresar una señal asíncrona al dominio de reloj interno. Una vez recibido un byte válido, este se almacena y se presenta en `LED[7:0]`.

IV-D7. Validación: La verificación funcional se realizó de dos formas. Primero, mediante un testbench que ejercitó tanto el transmisor como el receptor, enviando y recibiendo bytes conocidos. Segundo, mediante validación física con una terminal serial en Windows, específicamente PuTTY.

En la práctica se comprobó que al presionar el botón el sistema enviaba correctamente el mensaje “Hola mundo”.

Asimismo, se verificó que el receptor aceptaba caracteres alfabéticos, numéricos, espacios y caracteres especiales provenientes del teclado, reflejando los datos recibidos en los LEDs de la tarjeta.

V. RESULTADOS Y VALIDACIÓN

Los resultados obtenidos demostraron el correcto funcionamiento de los cuatro diseños desarrollados. La simulación permitió comprobar la validez lógica de cada módulo antes de su implementación física, mientras que la etapa de programación de la FPGA permitió confirmar el comportamiento real del hardware.

En el ejercicio del complemento a 2, la salida observada coincidió con la transformación esperada para los patrones de entrada probados. En el caso del multiplexor parametrizable, las pruebas para anchos de 4, 8 y 16 bits evidenciaron una selección correcta en los tres escenarios evaluados. Para el generador PWM, la variación del brillo del LED confirmó el cambio del duty cycle en función del valor de los switches. Finalmente, el sistema UART permitió verificar tanto la transmisión del mensaje predefinido como la recepción de datos desde una terminal serial externa.

La combinación entre simulación y validación física permitió confirmar que las descripciones HDL fueron correctas, sintetizables y coherentes con los requerimientos establecidos en el laboratorio.

V-A. Análisis de Recursos de Implementación

Los reportes de utilización obtenidos en Vivado mostraron un comportamiento coherente con la arquitectura de cada módulo desarrollado. En el caso del complemento a 2 y del multiplexor parametrizable, la implementación se realizó esencialmente con LUTs, al tratarse de lógica combinacional pura. En contraste, el generador PWM y el sistema UART requirieron recursos secuenciales adicionales, principalmente flip-flops, debido al uso de contadores, registros y máquinas de estados.

Desde una perspectiva de implementación, todos los diseños presentaron una ocupación baja de los recursos disponibles de la FPGA, lo cual demuestra que las soluciones propuestas no solo cumplen funcionalmente con los requerimientos del laboratorio, sino que además resultan eficientes en términos de síntesis. De igual forma, no se detectó inferencia de latches en los módulos donde no se esperaba memoria, confirmando que el código HDL fue estructurado correctamente.

VI. DISCUSIÓN

El laboratorio permitió comparar de manera práctica distintas formas de modelado digital dentro de un mismo flujo de diseño. Los ejercicios 1 y 2 mostraron con claridad el comportamiento de sistemas combinacionales, en los cuales la salida depende exclusivamente del valor presente en las entradas. En estos casos, el uso de asignaciones continuas y bloques `always_comb` resultó apropiado para describir hardware sin memoria.

Por otra parte, los ejercicios 3 y 4 evidenciaron la necesidad de utilizar lógica secuencial cuando el comportamiento del sistema depende del tiempo, del reloj o de estados internos. Esto se reflejó en el uso de contadores, registros, comparadores temporizados y máquinas de estados finitos.

Uno de los aspectos más relevantes del laboratorio fue la verificación de que la forma de escribir el HDL influye directamente sobre el hardware generado por la herramienta de síntesis. La ausencia de latches no deseados y la correcta separación entre lógica combinacional y secuencial constituyeron indicadores importantes de calidad del diseño.

Además, el uso de parametrización en el multiplexor demostró una ventaja significativa desde el punto de vista de reutilización y escalabilidad. De igual forma, el sistema UART permitió abordar aspectos más cercanos a una aplicación real, incluyendo sincronización, filtrado de rebote, control de temporización y comunicación con herramientas externas de software.

VII. CONCLUSIONES

El desarrollo de este laboratorio permitió consolidar conocimientos fundamentales sobre el diseño digital basado en HDL y el uso de herramientas modernas de automatización de diseño electrónico.

A lo largo de los ejercicios se logró comprender con mayor claridad la diferencia entre lógica combinacional y lógica secuencial, así como la relación directa entre la descripción escrita en Verilog/SystemVerilog y la estructura de hardware generada durante el proceso de síntesis. Esta relación evidenció que pequeñas decisiones en el código pueden tener un impacto significativo en el hardware final.

También se fortalecieron habilidades relacionadas con simulación, verificación funcional, implementación en FPGA y validación experimental en hardware real. En particular, la experiencia con PWM y UART permitió extender el aprendizaje más allá de circuitos puramente lógicos, incorporando conceptos de temporización, sincronización y comunicación serial.

No obstante, durante el desarrollo del laboratorio se identificaron aspectos que pueden mejorarse. En algunos casos, la validación inicial se realizó principalmente en hardware, lo cual limita la capacidad de detectar errores de forma temprana. Esto evidencia la importancia de priorizar la verificación en simulación antes de la implementación física, especialmente en diseños más complejos.

Además, se observó que el uso de recursos del dispositivo no siempre fue considerado desde una perspectiva de optimización. Es fundamental diseñar soluciones que no solo cumplan funcionalmente, sino que también sean eficientes en términos de utilización de recursos, evitando implementaciones innecesariamente costosas en hardware.

En términos generales, el laboratorio evidenció la importancia de desarrollar código sintetizable, modular y bien estructurado, así como la necesidad de adoptar un enfoque más riguroso en la validación y optimización del diseño para lograr sistemas más robustos y escalables.

VIII. RETROALIMENTACIÓN DEL PROFESOR Y MEJORAS IMPLEMENTADAS

Durante el desarrollo del laboratorio se identificó la necesidad de mejorar la organización y el seguimiento del trabajo realizado. Se recomienda implementar un control de versiones más estructurado, por ejemplo mediante el uso de *pull requests* asociadas a cada cambio, lo cual permite documentar mejor las decisiones de diseño y facilitar la revisión del código.

Asimismo, se detectó que algunos ejercicios fueron validados únicamente mediante pruebas en hardware. A partir de esta observación, se incorporaron verificaciones en software utilizando testbench más completos, lo cual mejora la confiabilidad del sistema y permite detectar errores antes de la implementación física.

Otro aspecto relevante es la consistencia en el uso del lenguaje de descripción de hardware. Se recomienda mantener una única línea de desarrollo, privilegiando el uso de SystemVerilog sobre Verilog tradicional, ya que ofrece mejoras en claridad, tipado y verificación. En este contexto, se realizó la transición del uso de `wire` hacia `logic`, lo cual permite una descripción más uniforme y moderna del diseño.

Y una recomendación que no es cuanto menos importante, el uso de la función `random` no es conveniente porque presenta una tendencia a repetir un valor, por lo que preferiblemente el uso de la función `$random` si cumple con una "aleatoriedad" mejor.

Adicionalmente, se reconoce la importancia de justificar las decisiones de diseño, no solo a nivel funcional, sino también en términos de eficiencia y buenas prácticas. Esto incluye evitar ambigüedades en la descripción del hardware, asegurar la ausencia de latches no deseados y estructurar adecuadamente los módulos para facilitar su reutilización.

En general, la retroalimentación recibida permitió identificar áreas de mejora tanto en la implementación como en la documentación del proyecto, contribuyendo a un desarrollo más ordenado, verificable y alineado con prácticas actuales de ingeniería digital.

IX. REFERENCIAS

REFERENCIAS

- [1] P. P. Chu, *FPGA Prototyping by SystemVerilog Examples*. Wiley, 2012.
- [2] D. Harris and S. Harris, *Digital Design and Computer Architecture*. Morgan Kaufmann, 2022.
- [3] S. Sutherland, S. Davidmann and P. Flake, *SystemVerilog for Design*. Springer, 2006.
- [4] D. Thomas, *Logic Design and Verification Using SystemVerilog*. CreateSpace, 2016.